LOVELY PROFESSIONAL UNIVERSITY

CSB205: SYSTEM DESIGN

Minor Project

Topic: Student Attendance System

Name: Raparthi Jashwanth
Reg .no: 12406316
Section: K24BX

Student Attendance System: Engineering Design Document (EDD)

1. Introduction and Project Scope

The Student Attendance System (SAS) is a critical application designed to modernize and streamline the process of recording and monitoring student attendance across K-12 and higher education institutions. The system aims to replace outdated manual processes with a reliable, efficient, and scalable digital platform that ensures data integrity and provides timely communication to stakeholders, specifically parents and administrators.

1.1 Scope Definition

The SAS will cover the primary functions of marking attendance, generating compliance reports, and dispatching automated alerts for absences.

In-Scope:

- Web and Mobile interfaces for teachers/staff to mark attendance.
- Real-time processing of attendance records.
- Asynchronous parent notification (SMS/Email) for unexcused absences.
- Administrative dashboard for student/class management and report generation.

Out-of-Scope (Phase 1):

- Integration with grading or academic systems.
- Biometric or RFID-based attendance capture.
- Complex payroll or substitute teacher management features.

---

2. Requirements Pack

2.1 Stakeholder Analysis & Prioritization

The prioritization of requirements is derived directly from the primary needs and power/interest matrix of the key stakeholders.

Stakeholder	Primary Need	Interest	Power	Priority

School Administrator	Compliance, Audit Trails, Aggregate Reports	High	High	P1 (Data Integrity)
Teacher	Speed, Ease of Use, Minimal Disruption	High	Low	P2 (UX/Latency)
Parent/Guardian	Timely Absence Notification, Safety Assurance	Medium	Low	P3 (Notification Reliability)
IT/Support Staff	System Stability, Observability	Medium	Medium	P4 (Resiliency)

2.2 Functional Requirements (FRs)

ID	Requirement Description	Priority	Responsible Service
FR-1.1	The system MUST allow teachers to mark a student's status as Present, Absent, or Late for a specific session/period.	High	Attendance
FR-1.2	The system MUST support bulk marking of attendance for an entire class roster.	High	Attendance
FR-2.1	The system MUST automatically trigger an alert (SMS/Email) to the	Critical	Notification

	parent for any unexcused absence marked by a teacher.		
FR-3.1	The system MUST generate aggregated attendance reports (Daily, Weekly, Monthly) filtered by class, student, or status.	High	Reporting
FR-4.1	Administrators MUST be able to manage (CRUD) student profiles, class rosters, and teacher assignments.	Medium	Auth/Admin
FR-5.1	The system MUST display a summary dashboard of current daily attendance statistics for administrators.	Medium	Reporting

2.3 Non-Functional Requirements (NFRs)

Category	**NFR Description**	**Target**	**Rationale**
Availability	System uptime during core school hours (7 AM - 4 PM)	99.9% (approx. 43 min downtime/month)	Ensure core functionality is available during peak marking times.
Performance	API response time	< 200ms (P95)	Support rapid

	for attendance marking (POST /batch)		marking by teachers to save class time.
Resilience	Notification delivery reliability	99.99%	Critical for student safety and parental communication.
Security	Compliance with FERPA/GDPR for student record protection	Mandatory	Legal requirement; all data encrypted at rest and in transit.
Scalability	Support 10x growth in simultaneous users (districts/schools)	100,000+ Students	Future-proofing for multi-district deployment.

2.4 Constraints & Assumptions

Constraints:

1. **Low-Bandwidth Support:** The mobile application MUST be designed with an offline-first capability to allow teachers to record attendance even in areas with poor or intermittent network connectivity.
2. **Budget Cap:** Infrastructure costs for Phase 1 are capped at a moderate tier, necessitating the use of cost-effective managed services and optimized database usage (e.g., maximizing Redis for hot reads).

Assumptions:

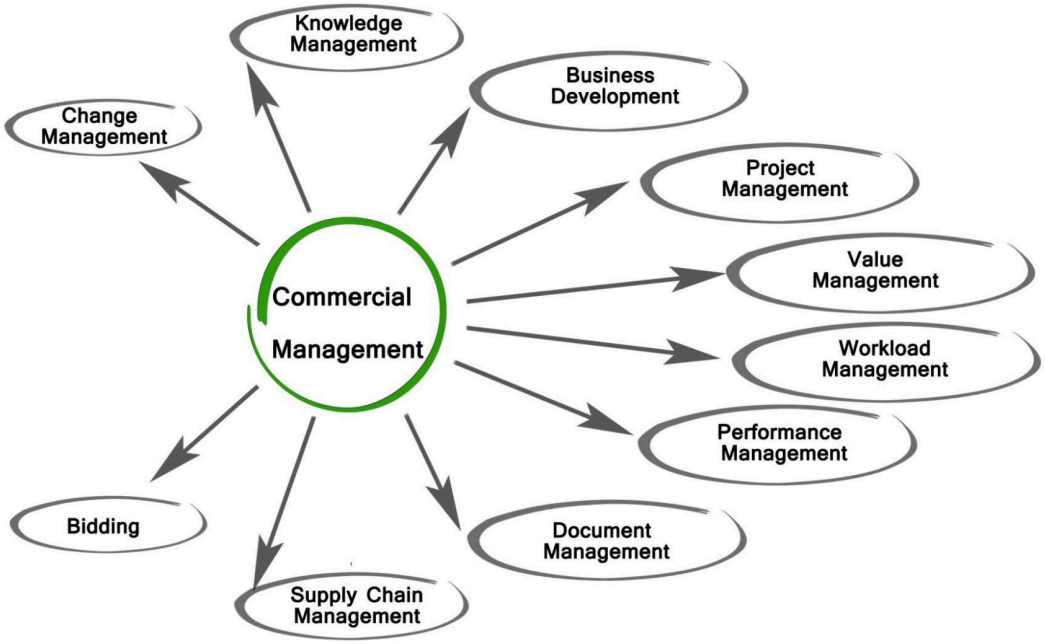
1. **Data Quality:** All student, teacher, and class schedule data imported initially is accurate and complete.
2. **Peak Load:** The highest transaction volume occurs between 8:00 AM and 8:45 AM local time, dictating auto-scaling rules.
3. **Parent Contacts:** Parents have provided at least one valid mobile number or email address for notification purposes.

---

3. Diagrams and User Flows

3.1 System Context Diagram

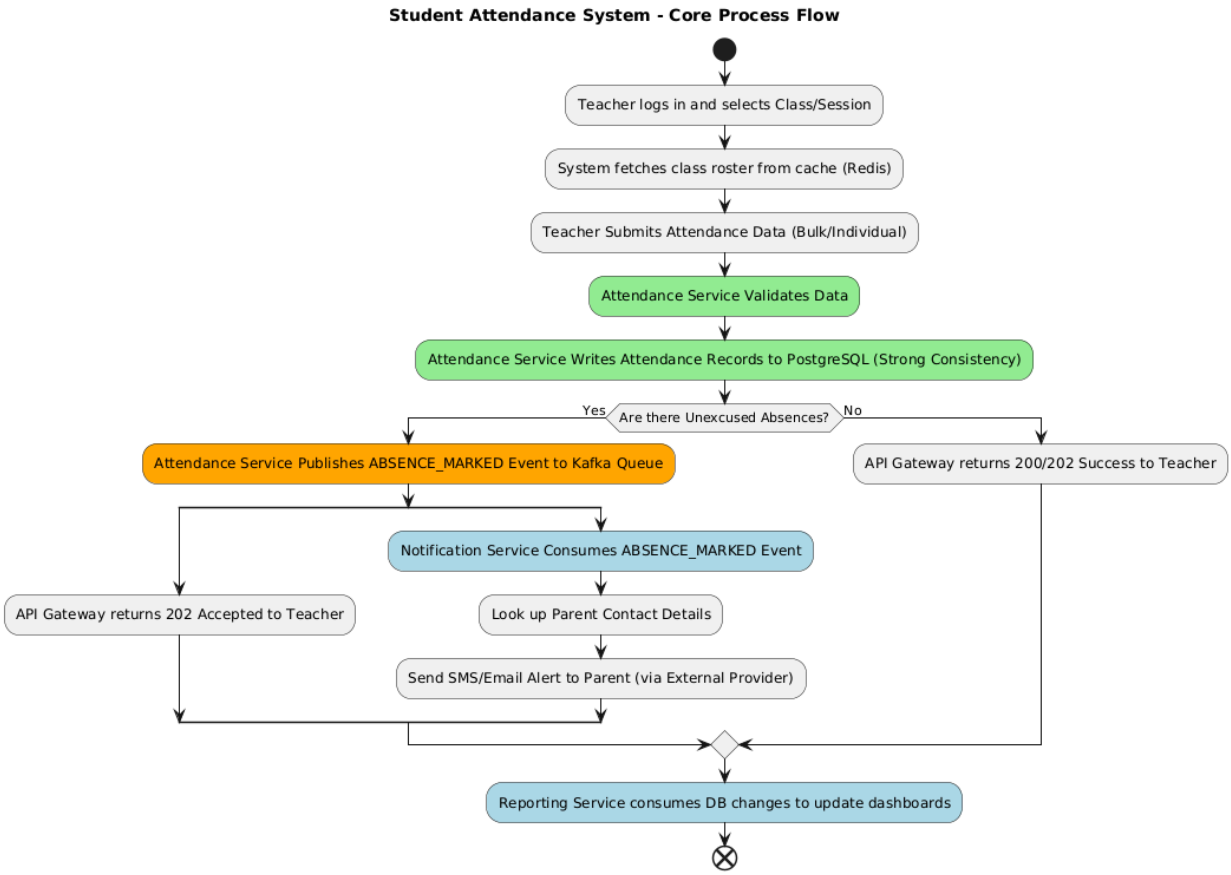
The System Context Diagram identifies the system's boundary and its interactions with external entities.



Getty Images

External Entities:

1. **Teacher/Staff Client:** The web or mobile app used for data entry.
2. **Administrator Client:** The web interface for configuration and reporting.
3. **Parent Notification Service (External API):** Third-party SMS/Email provider.
4. **District Data System (External):** Source for initial student/schedule data sync.



3.2 Use-Case Diagram

The Use-Case Diagram illustrates the primary functional requirements from the perspective of the actors.

Key Use Cases:

- **Teacher:** Mark Attendance, View Class Roster, Submit Absence Excuse.
- **Administrator:** Manage User Accounts, Generate Compliance Report, Audit Attendance History.
- **System (Automated):** Dispatch Absence Alert, Sync Student Data, Generate Daily Summary.

3.3 Core User Stories

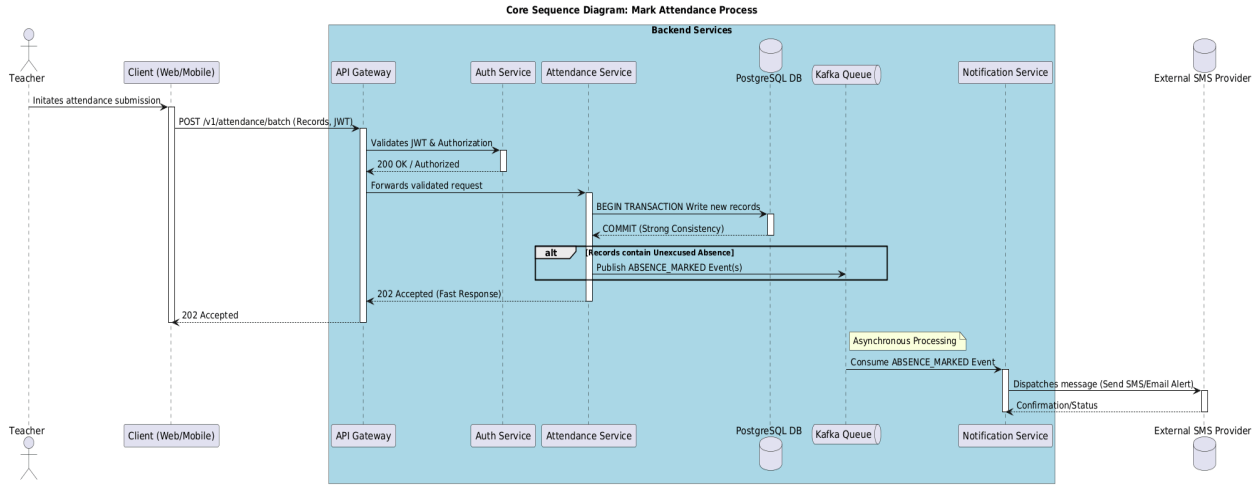
Role	Action	Goal	Rationale
Teacher	I want to view my current day's class roster immediately upon login,	so that I can quickly start marking attendance and minimize class time disruption.	High Priority (P2) for UX.
Parent	I want to receive a text message within 5 minutes of my child being marked absent,	so that I can promptly verify their safety and status.	Critical (P3) for safety assurance.
Admin	I want to run a report showing all students with more than 5 unexcused absences this month,	so that I can initiate truancy follow-up procedures.	High Priority (P1) for compliance.

3.4 Core Sequence Diagram: Mark Attendance Process

This sequence details the high-frequency path for a teacher submitting attendance and the system's reaction.

1. **Client** $\rightarrow$ **Gateway:** Teacher submits POST /v1/attendance/batch
2. **Gateway** $\rightarrow$ **Auth Service:** Validates JWT and authorization (is user a teacher for this class?).
3. **Gateway** $\rightarrow$ **Attendance Service:** Forwards the validated request.
4. **Attendance Service** $\rightarrow$ **PostgreSQL:** Begins transactional write for the new attendance records.
5. **PostgreSQL** $\rightarrow$ **Attendance Service:** Confirms write commitment (Strong Consistency).
6. **Attendance Service** $\rightarrow$ **Kafka Queue:** Publishes an ABSENCE_MARKED event for each student marked absent.
7. **Attendance Service** $\rightarrow$ **Gateway:** Returns 202 Accepted response to the client. (Fast response achieved by offloading notification).
8. **Notification Service** $\leftarrow$ **Kafka Queue:** Asynchronously consumes the ABSENCE_MARKED event.
9. **Notification Service** $\rightarrow$ **External SMS Provider:** Dispatches the

communication.



---

4. High-Level Architecture (HLA)

The system employs a microservices architecture to ensure independent scaling, deployment, and failure isolation.

4.1 Service Breakdown

Service	Primary Function	Data Store/Dependency
API Gateway	Authentication, Rate Limiting, Request Routing	Auth Service
Auth Service	User/Role management (Teacher, Admin), JWT generation	PostgreSQL
Attendance Service	Core business logic, attendance CRUD operations	PostgreSQL, Redis (Cache)
Notification Service	Asynchronous consumption of events, communication dispatch (SMS/Email)	Kafka, External SMS/Email Providers

Reporting Service	Reads from PostgreSQL and potentially read-replicas; generates aggregated reports	PostgreSQL Read-Replica

4.2 Data Stores, Cache, and Queue

- **Primary Data Store (PostgreSQL):** Selected for its transactional integrity (ACID properties) and relational strength. It is essential for storing core relational data (Students, Classes, Attendance Records) where strong consistency is paramount.
- **Cache (Redis):** Used for caching high-read, low-write data such as daily class rosters, teacher schedules, and session tokens. This reduces load on PostgreSQL during peak times.
- **Message Queue (Kafka):** Provides reliable, persistent, and decoupled communication between the Attendance and Notification Services. This ensures that a failure in the SMS provider does not block attendance marking.
- **CDN (Content Delivery Network):** Used to host and serve static front-end assets (HTML, CSS, JavaScript, React bundles) for the web application, improving load times globally.

---

5. Engineering Design & Notes

5.1 Capacity Planning & Back-of-the-Envelope Sizing

Assumptions for Sizing: 10,000 students, 2 school sessions per day, 200 school days per year.

Metric	Calculation	Result	Implication
Daily Writes	10,000 students $\times$ 2 marks/day	20,000 records/day	Low write volume overall.
Peak RPS (Write)	20,000 writes / (45 minutes $\times$ 60 seconds)	≈ 7.4 RPS average (sustained)	The peak is low, but spikes are expected. System must handle **500 RPS burst** due to

			simultaneous teacher submissions.
Annual Storage	1KB/record $\times$ 20,000 $\times$ 200 days	≈ 4 GB/year	Storage is inexpensive; no immediate concern for sharding by volume.
DB Performance	Requires low latency indexing for teacher lookups and attendance insertion.	99.9% of attendance writes must be $< 100\text{ms}$.	

5.2 API Specifications (Example)

Endpoint: POST /v1/attendance/batch

- **Description:** Allows a teacher to submit attendance records for multiple students in a class.
- **Body (JSON):**

```
{
  "classId": "CS_101",
  "sessionId": "AM_20251120",
  "teacherId": "T_54321",
  "records": [
    {"studentId": "S_1001", "status": "PRESENT", "isExcused": false},
    {"studentId": "S_1002", "status": "ABSENT", "isExcused": false},
    {"studentId": "S_1003", "status": "LATE", "isExcused": true}
  ]
}
```

- **Response:** 202 Accepted (Indicates processing has begun and notifications are queued).

5.3 Data Model (Conceptual)

The relational model centers around three core tables in PostgreSQL:

Table	**Primary Keys/Indices**	**Key Fields**	**Relationships**

Student	student_id (PK)	name, grade, parent_contact_id	1:N with Attendance_Log
Class	class_id (PK)	name, teacher_id, schedule	1:N with Attendance_Log
Attendance_Log	log_id (PK)	student_id (IDX), class_id (IDX), date, status, timestamp	N:1 with Student, N:1 with Class

Indexing: Mandatory composite index on (student_id, date) to support fast reporting queries.

5.4 Consistency Choices

- **Attendance Records (PostgreSQL): Strong Consistency**. It is critical that once a teacher marks attendance, that record is immediately accurate and available for reporting and notification checks.
- **Notifications (Kafka/Notification Service): Eventual Consistency (At Least Once)**. The notification is triggered asynchronously via the queue. The system guarantees the notification will be processed eventually, but not immediately after the database write is committed.

5.5 Caching & Indexing Strategy

- **Redis Caching:** TTL of 24 hours for class rosters and daily schedules. The cache is invalidated immediately upon administrative schedule changes.
- **Database Indexing:** Beyond the required composite index on (student_id, date), indices will be placed on foreign keys (teacher_id, parent_contact_id) to accelerate join operations in the Reporting Service.

5.6 Rate Limiting

The API Gateway will implement a **Token Bucket Algorithm** to protect the Attendance Service from bursts.

- **Limit:** 5 batch submissions per teacher (User ID) per minute.
- **Goal:** Protect against system abuse or accidental double-submissions, ensuring fair access during peak hours.

5.7 Resiliency (Retries, Timeouts, Circuit Breakers)

- **Timeouts:** Aggressive timeouts (e.g., 500ms) for internal service-to-service communication.
- **Retries:** Implemented with **Exponential Backoff** for external calls (e.g., Notification Service calling External SMS Provider). Retries are limited to 3 attempts before moving the message to a Dead Letter Queue (DLQ).

- **Circuit Breakers:** The Notification Service uses a Circuit Breaker (e.g., a Hystrix-like pattern) on its external dependencies (SMS/Email gateway). If the external service's error rate exceeds a threshold (e.g., 5% errors in a 60-second window), the circuit opens, requests are automatically failed quickly, and the queue is paused to prevent cascading failure.

5.8 Observability (Logs/Metrics/Traces)

- **Logs:** Centralized logging via a robust platform (e.g., ELK Stack or similar cloud-based service). All logs include trace_id, user_id, and service_name for context.
- **Metrics (Prometheus/Grafana):**
  - **Golden Signals:** Latency (P50, P95, P99 for all APIs), Traffic (RPS), Errors (Status codes), Saturation (DB connection pool usage, CPU).
  - **Business Metrics:** Daily Absence Count, Parent Notification Success Rate.
- **Traces (Jaeger/Zipkin):** Distributed tracing across all services is mandatory, especially the Mark Attendance and Generate Report paths, to diagnose latency bottlenecks.

5.9 Maintenance Plan

- **Deployments:** Utilize Blue/Green deployment strategy across all microservices for zero-downtime releases.
- **Database:** Daily automated backups of PostgreSQL to cold storage. Quarterly review of query execution plans.
- **Data Retention:** Attendance records older than seven years will be archived to low-cost archival storage, maintaining only aggregated statistics in the primary DB for long-term reporting.

---

6. Quality Targets & Trade-off Discussion

6.1 Explicit Service Level Objectives (SLOs)

Metric	Service	SLO Target	SLI (Service Level Indicator)
Availability	Overall System	99.9% Uptime (Monthly)	Percentage of successful requests over total requests.
Latency	Attendance Marking API	P95 $< 200\text{ms}$	Time between request receipt at

	(/batch)		Gateway and response return.
Freshness	Student Absence Notification	< 5 minutes	Time between teacher marking 'Absent' and message dispatch.
Correctness	Attendance Record Integrity	100%	Percentage of attendance records matching the original submission data.

6.2 Scalability Plan (Sharding/Partitioning)

While initial capacity planning indicates PostgreSQL can handle the load, planning for future expansion is necessary.

- **Database Sharding Strategy:** Once the system supports more than 100,000 students or 20 large school districts, sharding will be introduced. The proposed sharding key is **School_ID**.
    - **Rationale:** All core operations (marking, reporting, and management) are almost always scoped to a single school. This sharding key ensures that cross-shard joins are minimized, maintaining high performance as the system scales across multiple instances (shards).
- **Horizontal Scaling:** All services (Auth, Attendance, Notification, Reporting) are designed to be stateless (outside of the database connection pool) and will be deployed in containerized environments (Kubernetes) to facilitate rapid horizontal scaling based on resource utilization metrics (CPU/Memory).

6.3 Trade-off Discussion

6.3.1 Strong vs. Eventual Consistency

Choice	Impact	Rationale for Selection
Attendance Write: Strong Consistency (PostgreSQL)	Ensures that a critical record, once marked, is immutable and instantly available for audit and reports.	**Critical for Legal Compliance and Audit.** A parent notification cannot be sent based on a potentially stale record.

Notification Dispatch: Eventual Consistency (Kafka)	Allows the high-frequency marking API to return quickly, isolating it from the slow/unreliable external SMS service.	**Critical for Performance.** A teacher should not wait for an SMS provider's network delay.

6.3.2 SQL vs. NoSQL Primary Data Store

- **Decision:** PostgreSQL (SQL)
- **Justification:** While NoSQL (like Cassandra or MongoDB) offers higher write throughput, the core use case requires complex joins (e.g., "Find all students in Class X whose Teacher Y has marked them Absent more than Z times"). PostgreSQL's ACID properties guarantee the relational integrity crucial for student record management and government compliance reporting (P1 requirements). The write load is sufficiently low that PostgreSQL can handle it without issue.

---

7. Conclusion

The Student Attendance System design employs a robust microservices architecture grounded in strong data integrity principles. By utilizing PostgreSQL for transactional consistency and Kafka for asynchronous notification reliability, the system is engineered to meet the stringent performance (P95 $< 200\text{ms}$) and availability (99.9%) targets required for an educational setting. The defined SLOs, coupled with the detailed maintenance and resiliency plans (Circuit Breakers, Rate Limiting), ensure a highly scalable and maintainable platform ready for multi-district deployment.
